

In one line: A modifier **wraps** the view in a generic type (`ModifiedContent<Content, M>`) — static structure, so the wrap keeps identity. Reuse has three shapes: a **ViewModifier** (a look that may own state), a **style** (a control's look *without* losing its behaviour, inherited through the environment) and, since iOS 18, a **custom container** that lays out the *resolved subviews* of its content.

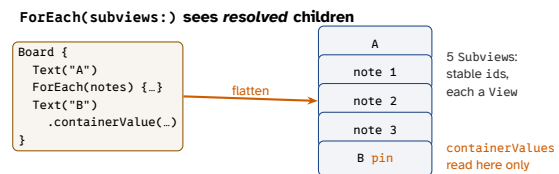
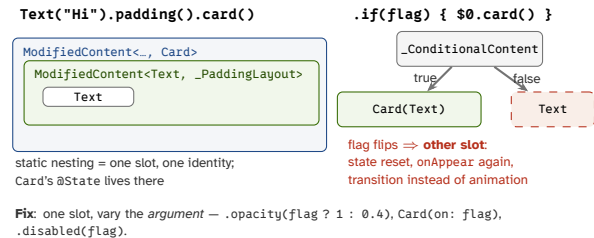
How it works

- **ViewModifier:** `body(content:);` may hold `@State` / `@Environment` like a view. Publish via extension `View { func card() -> some View { modifier(Card()) } }`. A stateless helper can just be the extension.
- **Content parameters:** `struct Board<Content: View> + @ViewBuilder var content: Content (or init(@ViewBuilder content:) that calls it). Store the built view, not the closure — closures never compare equal, so the child always re-runs.`
- **Styles:** `ButtonStyle.makeBody(configuration:)` gets label, `isPressed`, role — tap, disabled state and accessibility stay the system's. **PrimitiveButtonStyle:** you own the interaction, call `configuration.trigger()`. Also `ToggleStyle($configuration.isOn)`, `LabelStyle(title, icon)`.
- **Propagation:** `.buttonStyle(.pill) / .toggleStyle(.switch)` on a *container* styles every control below (environment; nearest wins). Sugar: extension `ButtonStyle` where `Self == Pill { static var pill: Pill { .init() } }`.
- **Custom containers (18):** `ForEach(subviews: content)` or `Group(subviews: content) { subs in }` — a `SubviewsCollection` (count, [i], slices; built on demand). Content is **resolved:** a `ForEach` of 5 = 5 subviews, Groups flattened. Subview = Identifiable View proxy, resolved in the *container's* environment; the child's own modifiers apply first. `ForEach(sections:)` → `SectionConfiguration` (header / content / footer).
- **Container values (18):** `@Entry` in extension `ContainerValues`; child `.containerValue(\.isPinned, true)`; container reads `sub.containerValues.isPinned`. **Direct** container only, no merge, never escapes (unlike preferences).
- **@Entry** (Xcode 16 macro, back-deploys): key boilerplate for `EnvironmentValues`, `Transaction`, `ContainerValues`, `FocusedValues` (focused: Optional, no default).
- **Type shapes:** some `View` / generic `Content` = one static type the diff can see through; `AnyView` = runtime box, a new wrapped type replaces the subtree. `.equatable()` lets your `==` skip body.

Traps

- ! The popular `.if(cond) { $0.mod() }` extension = `_ConditionalContent`: identity flips with the flag — lost state, broken animations. *Vary arguments.*
- ! `onTapGesture` on a styled `Text` instead of a `Button` — no button trait, no disabled state, no keyboard / pointer. Restyle with `ButtonStyle`.
- ! Own gesture inside a `ButtonStyle` label fights the system tap — `PrimitiveButtonStyle + trigger()`.
- ! A style on a Subview may do nothing: it can already be a styled leaf.
- ! `AnyView` from a protocol so “types line up” — use `associatedtype Body: View / @ViewBuilder`.
- ! Pre-18 containers via `_VariadicView` — underscored, unsupported.

Wrap vs branch



Example — modifier, style, container

```
struct Card: ViewModifier {
    @Environment(\.colorScheme) private var scheme
    func body(content: Content) -> some View {
        content.padding(12)
        .background(.background, in: .rect(cornerRadius: 12))
        .shadow(radius: scheme == .dark ? 0 : 4) } }
extension View { func card() -> some View { modifier(Card()) } }
struct Pill: ButtonStyle { // look only
    func makeBody(configuration: Configuration) -> some View {
        configuration.label.padding(8).background(.tint, in: .capsule)
        .opacity(configuration.isPressed ? 0.6 : 1) } }
extension ContainerValues { @Entry var isPinned = false }
struct Board<Content: View>: View { // iOS 18
    @ViewBuilder var content: Content // built view
    var body: some View {
        VStack { ForEach(subviews: content) { sub in
            sub.card().overlay(alignment: .topTrailing) {
                if sub.containerValues.isPinned { PinBadge() } }
            } } } }
```

Remember

“Wrap, don’t branch. A style keeps behaviour. A container reads resolved children.”